

Vision-Based Drone Landing on a Moving Platform

Detailed project report (version 5 of the work): problem, method, implementation, every experiment, findings, file map, commands to reproduce and watch the landings, and viva preparation

Yashwanth · Master's Thesis Project (MTP) · status as of 6 October 2026 · repository: github.com/yashwanth73337/drone-landing-rl (folder v5_shin/)

How to read this report. Section 1 states the problem. Sections 2–5 explain the background, the method being reproduced and how it was implemented, from first principles; every symbol is defined in the glossary (Section 0). Sections 6–8 list what was done, what went wrong and how it was fixed, and every result. Section 9 gives the conclusions and Section 10 the next steps. Sections 11–12 give every file with its path and purpose, and where and in what form each result is stored. Section 13 gives the commands to reproduce the results and to watch each method land. Section 14 is a set of viva questions with full answers, from basic to advanced. Unless stated otherwise, success rates are measured on the same 1,000 fixed test episodes for every method.

Summary in plain words.

- **Goal.** A drone must land on a moving vehicle using only its own camera and its own motion sensors, with no GPS and no help from the vehicle.
- **Question from the supervisor.** When such a drone fails, is it because it does not know where the platform is (a *state*, or estimation, problem) or because it does not know how to fly to it even when it knows (a *control* problem)?
- **Approach.** A faithful re-implementation of a 2026 journal method (Shin et al., IEEE Robotics and Automation Letters) in simulation, trained twice: once with the true platform state given to the policy (the "privileged" variant), once with only the camera (the "vision" variant). Then targeted tests separate the two causes.
- **Main findings.** (1) With the true state, the learned controller lands 97–100% of the time up to moderate platform motion and 85% at full motion (three training runs). (2) Vision lands as well as the true-state controller *at the same stage of training* on the motion it has trained on, but it needs about 7–10 times more training experience to get there, it cannot learn at all without an extra estimation objective, and it generalises worse to faster, unseen platform motion. (3) Correcting the vision policy's explicit position/velocity estimate at test time fixes only about 13% of its shortfall. (4) Every learned policy lands too hard (2–4 m/s); the cause is the task definition plus a property of the simulated motors, and the paper's success measure hides it.
- **So, state or control?** The difficulty is on the state side, but it shows up as slow and fragile *learning* of state from images, not as a lower performance ceiling once learned.

0. Glossary of terms and symbols

Every abbreviation and symbol used in this report, in plain words.

Term or symbol	Meaning
Drone, quadrotor	A flying robot with four propellers. The simulated model is the LMF2 drone from the AerialGym simulator: mass 1.24 kg, body 0.5 m wide.
Platform, pad	The moving ground vehicle and the 1.5 m × 1.5 m landing surface on top of it (top at 1.0 m height).
Policy	The function that decides the drone's command from what it observes. Here a neural network.
Reinforcement learning (RL)	Learning a policy by trial and error: the policy acts, receives a numerical reward, and is adjusted to collect more reward.
Episode	One landing attempt from start to finish (at most 300 decisions = 30 seconds).
Step, time step	One decision of the policy, every 0.1 s (10 decisions per second). Inside one step the physics is simulated 10 times (every 0.01 s).
Environment step count	Total number of decisions taken during training, summed over the 16 parallel simulations. Training budgets are 2 or 4 million.
MDP / POMDP	Markov decision process: the standard mathematical model for RL. Partially observable MDP: the policy cannot see the full state (here, the platform's motion) and must infer it.
PPO	Proximal Policy Optimisation, the RL algorithm used. It improves the policy in small, safe steps (Section 3.4).
CNN	Convolutional neural network: the part of the network that turns the camera image into numbers.
LSTM	Long short-term memory: a recurrent network layer that keeps a memory across time steps, so the policy can remember where the pad was when it is out of view.
MLP	Multi-layer perceptron: a plain feed-forward neural network.
Critic, value function V	A second network used only during training. It predicts how much future reward to expect; PPO uses it to judge whether an action was better or worse than expected.
Asymmetric (privileged) critic	The critic is allowed to see the true platform state during training, while the policy is not. The critic is thrown away after training, so the final drone never uses privileged information.
I_t	The grayscale camera image at step t.
u_t	The drone's own motion measurement at step t: velocity in its body frame (3 numbers) and its orientation as a quaternion (4 numbers), both with sensor noise.
s_{rel}	Relative state of the platform, 6 numbers: Δx (platform position minus drone position, 3 numbers) and Δv (platform velocity minus drone velocity, 3 numbers), both expressed in the drone's body frame.
$\tilde{s}_{rel}$ (estimate)	The vision network's estimate of s_{rel} : the first 6 outputs of its estimation layer.
L_{est} (estimation loss)	Mean squared error between $\tilde{s}_{rel}$ and the true s_{rel} . Minimised during training alongside the PPO objective. Its weight λ_{est} is 1.0.
r_{active} (active-perception reward)	Extra reward $-0.1 \times \text{clip}(L_{est} \text{ at the next step} - 0.01, 0, 1)$: it penalises actions after which the estimate gets worse, encouraging the drone to keep the pad in view. $\alpha = 0.1$, $\beta = 1.0$, $\tau = 0.01$ are its constants.
a_t (action)	The command: forward speed v_x , sideways speed v_y , vertical speed v_z (all in m/s) and turning rate ω_z (rad/s), expressed in the heading frame (rotated only by the drone's heading, not tilted).
v_z	Vertical speed. Positive = upward, negative = downward. A touchdown at -2 m/s means hitting the pad while moving down at 2 m/s.
Δz	Height difference: pad top height minus drone centre height. Negative while the drone is above the pad.

Term or symbol	Meaning
c (difficulty)	Curriculum scalar between 0 and 1 that scales how fast and erratically the platform moves. $c = 1$ is the full task (platform up to 8 m/s, turning up to 30°/s). Level L corresponds to $c = L/80$: level 10 $\rightarrow c = 0.125$, level 40 $\rightarrow 0.5$, level 50 $\rightarrow 0.625$, level 80 $\rightarrow 1.0$.
Curriculum level	Training starts at level 10 and is promoted by 10 each time the drone succeeds in at least 80% of a window of 512 episodes.
Domain randomisation (DR)	Random variation of physics and appearance during training (controller gains, wind-like forces, initial motion, ground texture, lighting, colours) so that the policy does not over-fit to one exact world.
ArUco marker	A black-and-white square pattern printed on the pad, easy for a camera to see. Used here instead of the paper's PACMAN keypoint pad, whose trained detector is not publicly available.
Variant P	The privileged (true-state) policy: it receives the true s_{rel} and u_t , no camera. It shows the best a learned controller can do with perfect information: the control ceiling.
Variant A	The vision policy: camera + u_t , CNN + LSTM + estimation layer, trained with L_{est} and r_{active} . It is the paper's own "without keypoint encoder" configuration.
A-noEst	Variant A with the estimation loss switched off ($\lambda_{est} = 0$, which also switches off r_{active}). The paper's "without state estimation" ablation.
True-state injection	A test-time diagnostic: A's trained network is run as usual, but its 6-number estimate $\hat{s}_{rel}$ is replaced by the true s_{rel} before the decision layer.
Oracle	A hand-written controller that reads the true state from the simulator. Not a competitor; it only proves the task is solvable.
Success	Paper definition: any contact with the top surface of the pad.
Strict success	Success with the drone's centre over the pad (the 0.5 m drone body can touch the edge with its centre outside).
Safe success ($\leq X$ m/s)	Success with a touchdown vertical speed of at most X m/s, counted over all episodes.
Impact (touchdown) speed	The drone's vertical speed relative to the platform just before contact.
Pinned evaluation	Testing on the same 1,000 episodes every time: episode i uses random seed 9000 + i, so every method faces identical starting positions, platform motions and disturbances. The policy uses its average action (no exploration noise).
95% confidence interval	The Wilson score interval for a success rate measured on 1,000 episodes; roughly ± 2 –3 percentage points.
Seed	The starting number of a training run's random generator. Different seeds give different but equally valid training runs; the project rule is three seeds before a result is reported.
asym / sym_slow / sym_fast	Motor models: asymmetric (AerialGym's, motors spin up in 50–80 ms and down in 5 ms), symmetric slow (both directions 50–80 ms), symmetric fast (both 5 ms).
Checkpoint	A saved copy of the network weights during or after training (file ending .pt).
D1 ... D18	Numbered design decisions recorded in SPEC.md (Section 5.13).

1. Problem statement

1.1 The physical task

A quadrotor starts 2 to 8 metres above, and up to 3 metres to the side of, a ground vehicle that carries a 1.5 m $\times$ 1.5 m landing pad. The vehicle drives with a speed and a turning rate that change randomly every 0.1 seconds; at full difficulty its speed can reach 8 m/s (about 29 km/h) and it can turn at up to 30° per second. The drone does not know the vehicle's speed or route. It must touch down on the pad's top surface within 30 seconds.

The drone may use only two sources of information: (1) a grayscale camera fixed to its front and tilted 60° downward (512 $\times$ 320 pixels, 90° horizontal field of view), and (2) its own measured velocity and orientation, as an onboard inertial navigation system would give, including measurement noise (0.05 m/s and 0.5°). No GPS, no motion-capture system, and no communication with the vehicle

are allowed.

The camera often loses sight of the pad: the pad leaves the image when the drone is very close, when it turns, or when the vehicle accelerates. In a successful landing by the scripted oracle, the pad centre is out of the image in about half of all frames. The drone therefore has to remember and predict where the pad is.

1.2 The formal setting

The task is a partially observable Markov decision process. At every step t (every 0.1 s) the policy receives the observation $o_t = (l_t, u_t)$ and outputs the action $a_t = [v_x, v_y, v_z, \omega_z]$. A geometric flight controller turns that velocity command into motor thrusts. The hidden quantity that matters most is s_{rel} , the platform's position and velocity relative to the drone. The policy is trained to maximise the expected sum of discounted rewards (discount factor $\gamma = 0.99$).

Element	Definition in this project
Episode length	At most 300 steps of 0.1 s (30 s).
Start	Height above pad top 2–8 m (uniform), sideways offset -3 to $+3$ m in each direction, platform heading offset -60° to $+60^\circ$, drone facing the pad $\pm 15^\circ$ with the pad centre visible.
Success	Contact with the pad's top surface.
Failures	Crash into the ground; crash into the side of the platform; tilt beyond 80° ; drifting away (more than 15 m sideways or more than 12 m above the pad); timeout at 300 steps.
Difficulty c	Platform start speed uniform between 0 and $8c$ m/s; each step its speed changes by up to $\pm 0.5c$ m/s and its turning rate by up to $\pm 3c$ $^\circ$ /s.

1.3 The research question

When vision-based landing on a moving platform fails, is it a **state problem** (the drone does not know accurately enough where the platform is and how it moves) or a **control problem** (even with a good estimate, the learned policy, or the way it is trained, does not produce a landing)? The two answers call for different remedies: better perception and estimation, or a better controller, reward or training scheme. Earlier versions of this project (V1–V4, Section 2) could not separate them.

A second, related claim developed through the project: *binary success rate is a misleading measure for this task*. Two policies with the same success rate can fail in very different ways, and a counted "success" can be an impact that would damage a real drone.

2. Background: versions 1 to 4, and why version 5

The earlier versions used a small 27-gram drone model (Crazyflie) in the gym-pybullet-drones simulator, the TD3 reinforcement-learning algorithm, and the 15-number true-state observation of the Lander.AI paper. Each version changed one thing:

Version	What changed	What happened
Strand B	Faithful rebuild of Lander.AI with the true state	100% on a still pad, 98.7% at 0.25 m/s, three seeds. Kept as a frozen reference.
V1	A downward camera added, only observing	The pad is visible only about half of a landing; blind stretches cluster at the end.
V2	The frozen policies fly on ArUco detection plus dead reckoning	Vision costs about 45 percentage points; the drone descends onto where the pad used to be.
V3a	The policy is retrained on the vision observation	It learned to hover rather than land.
V3b	Dead reckoning replaced by an LSTM estimator trained offline	85% on a still pad; on a moving pad crashes returned and later training collapsed into hovering, even when given the true state.

Version	What changed	What happened
V4	The Shin et al. method, but with many deviations (position-offset actions, downward camera, different drone, constant platform speed)	The estimator worked, but the policy never landed.

Because V4 differed from the paper in so many ways, its failure could not be attributed to the method or to the deviations. **Version 5 therefore rebuilds the paper from scratch and as faithfully as possible**, with every unavoidable difference decided and written down before coding, and without reusing V1–V4 code.

3. The method being reproduced (Shin et al., 2026)

W. Shin, M. Kim, T. Park, G. Bae, S. Kim and H. Oh, "Vision-Based Autonomous Drone Landing on Moving Platforms With Uncertain Motion via Deep Reinforcement Learning", IEEE Robotics and Automation Letters 11(5), May 2026. Perception reference: T. Park et al., "PACMAN", Image and Vision Computing, 2025.

3.1 The network

The camera image goes through a CNN, which produces 512 numbers describing what is seen. These, together with the motion measurement u_t , enter an LSTM with 512 memory units, which carries information across time (essential when the pad is out of view). An estimation layer combines the CNN output, the LSTM memory and u_t into a 256-number vector y_t , whose first 6 numbers are trained to equal s_{rel} . Finally a decision layer maps y_t and u_t to the action. In the paper the CNN contains a frozen, pre-trained keypoint detector for its special hexagonal pad; in our base configuration it is a plain CNN trained from scratch on an ArUco pad, which is the paper's own "without keypoint encoder" ablation.

3.2 Training signals

- **Reinforcement learning (PPO)** on the reward of Section 3.3.
- **Estimation loss L_{est}** : the first 6 outputs of the estimation layer are pushed toward the true s_{rel} (known in simulation). This teaches the network what to look for in the image.
- **Active-perception reward r_{active}** : a penalty when the next step's estimate is worse, encouraging the drone to fly so that it keeps seeing the pad.
- **Asymmetric critic**: the value network sees the true s_{rel} during training to give better learning signals; it is discarded afterwards.

3.3 The reward

At each step: +10 for a successful landing, −10 for a crash or drifting away, otherwise the sum of shaping terms below plus r_{active} . Here d_{xy} is the horizontal distance to the pad centre, Δz the height difference and the bracket $[x]_a^b$ means x clipped to the interval from a to b .

Term	Formula (as printed in the paper)	Weight	Meaning
Lateral progress	$[d_{xy}(t-1) - d_{xy}(t)]$ clipped to $[-1, 1]$	1.0	Reward for getting horizontally closer to the pad.
Vertical progress	$[\Delta z(t-1) - \Delta z(t)]$ clipped to $[-1, 1]$, divided by $\max(d_{xy}, 1)$	1.0	Reward for getting vertically closer, worth more when already above the pad.
Vertical-speed penalty	$-[v_z + 0.5]$ clipped to $[0, \infty)$	0.5	As printed, this penalises anything slower than a 0.5 m/s descent (hovering, climbing). The paper's text says it should penalise fast descent; the two disagree (Section 5.8).
Undershoot penalty	$-(\Delta z \text{ if } \Delta z > 0, \text{ else } 0)$	1.0	Penalty for being below the pad top.

Term	Formula (as printed in the paper)	Weight	Meaning
Yaw-rate penalty	$- \omega_z $	2.0	Penalty for turning.

3.4 Proximal Policy Optimisation in brief

PPO collects a batch of experience with the current policy, estimates for each action an *advantage* (how much better it turned out than the critic expected, using generalised advantage estimation with $\gamma = 0.99$ and $\lambda = 0.95$), and then updates the policy to make good actions more likely. The update is "proximal": the ratio between new and old action probabilities is clipped to $[0.8, 1.2]$, so one update cannot change the policy too much. Because the vision policy has an LSTM, experience is processed in sequences of 32 steps with the stored memory at the start of each sequence.

3.5 Curriculum and domain randomisation

Training starts with a slowly moving platform (level 10, $c = 0.125$) and is promoted by 10 levels each time the policy succeeds in at least 80% of 512 consecutive episodes, up to level 80 ($c = 1$). Physical and visual properties are randomised as in the paper's Table II.

3.6 The paper's results (success at full difficulty, 10,000 episodes)

Configuration	Success	Position / velocity estimation error
Full method (keypoint pad)	97%	0.47 m / 0.59 m/s
ArUco + plain CNN (our base configuration A)	91%	0.95 m / 1.06 m/s
Without state estimation	73.99%	–
EKF + RL baseline (marker detector + Kalman filter + RL)	59.77%	1.33 m / 1.50 m/s

4. Our approach to the research question

4.1 The variants and what each one tests

Variant	What the policy receives	Purpose
Oracle (scripted)	True state, hand-written rules	Proves the simulated task is solvable within the command limits.
P (privileged, true state)	True s_{rel} and u_t ; no camera	The control ceiling: what this reward, this PPO and this curriculum can achieve with perfect information. If P fails, the problem is control/training.
A (vision, proposed)	Camera image and u_t ; learns its own estimate	The system under study.
A + true-state injection	A's network, with its estimate replaced by the truth at test time	If A's failures come from a wrong estimate, injecting the truth should fix them.
A-noEst	Camera and u_t , no estimation loss	Does explicit supervision of the state help learning at all?
Curriculum-matched P	P checkpoints saved when P reached the same level as A	Removes the unfairness that P finished the curriculum and A did not.
Motor variants	Same checkpoints, symmetric motor models	Explains the hard landings.

4.2 The logic of the comparisons

- If P lands and A does not, the difference comes from vision (state side).

- If giving A the true estimate closes the gap, the explicit estimate is the problem; if not, the problem lies in how A learned to act or in other information it extracts from images.
- If A-noEst fails where A succeeds, learning the state is essential to learning to land.
- Comparisons must be at the same training stage; otherwise the better-trained policy wins for a trivial reason.

4.3 Working rules followed throughout

- **Faithful first.** Reproduce the paper as written; record every choice where the paper is silent, ambiguous or impossible to match as a numbered decision with its reason.
- **Build in blocks, test every block** before building on it (178 automated tests at present).
- **Change one thing at a time** between any two runs that are compared.
- **Pinned evaluation:** the same 1,000 test episodes for every method; every number is reported with the checkpoint, the training step count and the difficulty it was measured at.
- **Three seeds** before a result is reported (decision D14).
- **Report more than the paper's success measure:** failure types, strict and safe success, touchdown speed, estimation error by how long the pad has been out of view.
- **No training run or parameter change without the project owner's approval.**

5. Implementation in detail

5.1 Simulator

PyBullet 3.2.7 physics at 100 Hz (every 0.01 s); the policy decides every 0.1 s, so each decision is held for 10 physics steps. The paper used AerialGym on Isaac Gym, which needs hardware we do not have; we therefore copied AerialGym's drone, controller and motor parameters into our own PyBullet environment (decisions D1 and D2).

5.2 Drone and motors

AerialGym's LMF2 drone: 1.2 kg body plus four 0.01 kg propellers = 1.24 kg; inertia diag(0.0134, 0.0144, 0.0138) kg·m²; collision body a 0.5 m cube; each motor gives 0.1–10 N, so total thrust is 3.3 times the weight. Each motor is a first-order lag on rotor speed with time constant 50–80 ms when speeding up and 5 ms when slowing down (AerialGym's values). The paper's controller gains equal AerialGym's LMF2 settings exactly, which is why this drone was chosen.

5.3 Flight controller

A Lee geometric controller (a standard controller on the rotation group): the velocity error is multiplied by gains K_v (2.7–3.3 horizontally, 1.3–1.7 vertically) to give a desired acceleration and thrust direction; the attitude error is corrected with gains K_θ (1.6–1.85 roll/pitch, 0.25–0.4 yaw) and K_ω . Gains are randomised per environment. Commands are in the heading frame, so a forward command always means "forward in the direction the drone faces", independent of tilt. There is no integral term, so a constant thrust shortfall becomes a constant velocity error, which matters in Section 8.2.

5.4 Platform

A 1.5 × 1.5 × 1.0 m box moving on the ground like a car (speed and turning rate, integrated at 100 Hz). Speed and turning rate change randomly every 0.1 s as in the paper's Table I; speed is limited to 0–8 m/s and turning rate to ±30°/s (decision D6). The ArUco marker (dictionary 4×4_50, number 0, black square 1.2 m with a white border) is printed on the top face.

5.5 Camera and image

Pinhole camera, 512×320 pixels, 90° horizontal and 64° vertical field of view, mounted 0.15 m ahead of the drone's centre and tilted 60° below the forward direction. The colour image is converted to grayscale with the standard luma weights and averaged down to 256×160 pixels for the network (decision D9). For training, images are rendered on the CPU (PyBullet's TinyRenderer) because each GPU-rendered environment needs about 250 MB of the 4 GB graphics memory (decision D18).

5.6 Observation, action and the privilege boundary

The environment always computes five items: the image, u_t (noisy), the critic input (noise-free u_t plus the true s_{rel} , 13 numbers), the estimation target (true s_{rel}) and s_{rel} itself. A fixed table decides what each policy may read: the vision policy reads only the image and u_t ; the privileged policy reads u_t and s_{rel} . Tests check numerically that the vision policy's output does not change when the hidden platform velocity changes. Actions in $[-1, 1]$ are scaled to ± 10 m/s horizontally, ± 1 m/s vertically (decision D16) and $\pm 60^\circ/\text{s}$ turning.

5.7 Termination

Contacts are checked after every 0.01 s physics step. Contact with the pad top (contact normal pointing up) is a success; contact with a side face or the ground is a crash; tilt beyond 80° is a crash; more than 15 m sideways or 12 m above the pad is drifting; 300 steps is a timeout. The touchdown velocity is read just before the contact step, because the contact solver zeroes it afterwards.

5.8 Reward choices

The printed vertical-speed penalty $-[v_z + 0.5]^+$ rewards descending at 0.5 m/s or faster and penalises hovering; the paper's text says it penalises fast descent. We checked our sign conventions (height positive upward, like AerialGym) and showed that no sign convention makes the printed formula match the text. We tested both readings: the printed one is kept (decision D3); the text reading made the policy stall at level 10 with 30.5% success (decision D17). Distances in the reward are measured in the world frame; Δz is pad top minus drone height (decision D4).

5.9 Network sizes

Part	Structure	Parameters
CNN encoder	5 convolution layers, stride 2, kernels 5-3-3-3-3, channels 32-64-64-128-128, ELU activations, then a linear layer to 512	2.90 million
LSTM	Input $512 + 7 = 519$, memory 512	2.12 million
Estimation layer	$1031 \rightarrow 512 \rightarrow 256$ (the first 6 outputs are the estimate)	0.66 million
Decision layer	$256 + 7 = 263 \rightarrow 256 \rightarrow 128 \rightarrow 4$ (mean of a Gaussian; a learned spread, initial log standard deviation -0.5)	0.10 million
Critic	$13 \rightarrow 256 \rightarrow 256 \rightarrow 1$	0.07 million
Privileged actor (P)	$13 \rightarrow 256 \rightarrow 128 \rightarrow 4$	small

5.10 Training settings (PPO)

Setting	Value
Parallel environments × steps per update	$16 \times 256 = 4,096$ decisions per update
Sequence length for the LSTM	32 steps
Epochs × minibatches per update	5×4
Discount γ , advantage smoothing λ	0.99, 0.95
Clip range	0.2
Learning rate	0.0003, decreasing linearly to 0 over the run
Value-loss weight, entropy weight, gradient-norm limit	0.5, 0, 1.0
Estimation-loss weight λ_{est}	1.0 (0 for A-noEst)
GPU memory saving	Each minibatch is split into micro-batches of 8 sequences and the gradients are summed (identical result, tested).
Timeouts	Treated as truncation: the value of the final state is used instead of zero.
Speed on the lab desktop	True-state policy about 1,500 decisions/s (2M in 25 min); vision about 56 decisions/s (4M in 20 h).

5.11 Evaluation

Each test episode i uses seed $9000 + i$ for every random element (start position, platform motion, disturbances, sensor noise and controller gains). The policy takes its mean action. Reported per run and difficulty: success with its 95% Wilson interval; strict success; counts of each failure type; touchdown vertical speed (median, 10th and 90th percentile); commanded vertical speed over the last 0.5 s; safe success at 0.5, 1.0, 1.5 and 2.0 m/s; for vision policies, the estimation error overall and by how many steps the pad has been out of view.

5.12 Diagnostics added during the project

- **Motor modes** (`--motor asym|sym_slow|sym_fast`): change only the motor time constants, for evaluation or training, with every random stream unchanged.
- **True-state injection** (`--inject-true-state`): replace the vision policy's estimate by the truth at test time.
- **Viewer** (`scripts/watch.py`): a live 3D window or a video of any method on the pinned episodes.

5.13 Decisions D1-D18

Decision	Choice	Reason
D1	Own environment on plain PyBullet	The common gym-pybullet-drones base computes the reward before checking termination, and is built around a different drone.
D2	AerialGym LMF2 drone	The paper's controller gains are exactly LMF2's.
D3, D17	Vertical-speed penalty as printed	Text reading tested and rejected (30.5% vs 81.5% success).
D4	$\Delta z = \text{pad top} - \text{drone height}$	Makes the "undershoot" term meaningful.
D5	Levels 10-80 in steps of 10, promote at $\geq 80\%$ of 512 episodes	Matches the paper's figure and timing.
D6	Platform speed ≤ 8 c m/s, turning $\leq 30^\circ/\text{s}$	The paper's random walk is otherwise unbounded.
D7	Torque disturbance ± 0.004 N·m	The paper's table prints ± 4000 , an obvious typo.
D8	Own recurrent PPO	The auxiliary loss, asymmetric critic and extra reward do not fit standard libraries.
D9	Render 512×320 , network input 256×160	Paper resolution, affordable network.
D10	$\lambda_{\text{est}} = 1.0$	Unspecified in the paper.
D11	Include variant P	Needed for the state-versus-control question.
D12	Start facing the pad $\pm 15^\circ$ with the pad in view	The paper requires the pad in view at the start.
D13	50 procedurally generated ground textures	The paper's textures are not available.
D14	Three seeds per reported configuration	Seed-to-seed variation is large.
D15	Paper's success definition, plus strict success	The 0.5 m body can touch the edge with its centre off the pad.
D16	Vertical command ± 1 m/s	Unspecified; with ± 3 m/s the policy dived at 4 m/s; the paper's figures show about 1 m/s.
D18	CPU rendering for vision training	16 GPU renderers do not fit in 4 GB next to training; same overall speed.

6. What we did, step by step

Date (2026)	Step	Outcome
29 Sep	Moved the project to the lab desktop (4 GB GPU, 20 CPU threads); fixed a version-control rule that had hidden V3b's estimator code.	Desktop reproduces earlier numbers.
30 Sep	Read the paper; wrote SPEC.md with every parameter and decisions D1-D15.	Specification agreed before coding.
30 Sep	Built and tested blocks 1-12 and 14 (drone, platform, camera, termination, oracle, randomisation, CNN, environment, reward, curriculum, network, PPO, evaluation).	160 tests; oracle 99% at full difficulty.
30 Sep	First true-state training run (vertical limit ± 3 m/s).	91% success but dives at -4.1 m/s.
30 Sep	Decision D16 (± 1 m/s); second run.	81.5% success; still hits at -2.3 m/s.
1 Oct	Tested the text reading of the vertical-speed penalty.	30.5%; rejected (D17).
1 Oct	Audited signs and the motor model against AerialGym's source code; built the motor diagnostic.	Same checkpoint, symmetric motors: touchdown $-2.30 \rightarrow -0.78$ m/s.
1 Oct	Trained with symmetric motors.	Still -1.93 m/s: the root cause is the task definition.
1 Oct	Built block 13 (active-perception reward).	173 tests.
1 Oct	First vision launch ran out of GPU memory; measured it; decision D18; 100k-step smoke run.	Vision pipeline works, 57 decisions/s.
1-3 Oct	Vision run A_s1, 4M steps.	Level 50 of 80.
3 Oct	Same-episode comparison of P and A; built true-state injection; symmetric motors for A.	Injection recovers little; A also relies on the motor effect.
3-4 Oct	True-state seeds 2 and 3; A-noEst (4M steps).	P stable; A-noEst never leaves level 10.
4-6 Oct	Vision seeds 2 and 3 with evaluations and injection.	Levels 50 and 30; injection recovers about 13%.
6 Oct	Curriculum-matched comparison; learning-speed measurement; viewer script; this report.	Vision equals P at the same stage on trained motion; learns 7-10 $\times$ slower.

7. Problems found along the way, and their fixes

Problem	What it would have caused	Fix
PyBullet computed the drone's inertia from its collision box	Inertia four times too large; wrong rotation dynamics	Load the inertia from the model file; a test checks it
GPU renderer drew blank frames, shifted textures, sheared the marker, and flickering surfaces (z-fighting)	Images without a usable marker	Renderer loaded first; no embedded textures; 768-pixel colour texture; single pad mesh; hidden ground collision shape
Touchdown speed read after the contact solver zeroed it	Impact speed reported as about zero	Read just before contact
Jittery commands make the drone sink (motor asymmetry)	Policies descend faster than commanded	Documented, tested and diagnosed (Section 8.2)
Vertical command limit unspecified (± 3 m/s)	Policies dived at 4 m/s	D16: ± 1 m/s
16 GPU-rendered environments plus training exceed 4 GB	Out-of-memory crash	D18: CPU rendering
Motor integration differs from AerialGym's default (Euler versus Runge-Kutta)	Small difference in dynamics	Measured effect below 0.06 m/s; recorded
Duplicate downloads of SPEC.md led to committing an older copy	Repository out of date	Unique file names and checksum checks for every delivered file

8. Results

All success rates below come from the same 1,000 test episodes (seeds 9000–9999), deterministic actions, the paper's motor model, and the final checkpoint `latest.pt` of each run unless stated.

8.1 The environment was validated before any learning

Check	Result
Flight controller	Holds a hover exactly at the minimum, nominal and maximum gains; follows speed steps; reaches 9.5 m/s within 1 s.
Platform and camera	Motion statistics match the paper's tables (Kolmogorov–Smirnov tests); detected marker corners lie within 0.7–1.6 pixels of the geometric prediction.
Scripted oracle (true state)	99% success at full difficulty , touchdown about -0.6 m/s. The task is solvable and gentle landing is physically possible.
Visibility	During the oracle's landings the pad centre is out of the image in about half of all frames.
Network and PPO	The privilege boundary, gradient routing, recurrent replay and gradient accumulation are verified numerically.

8.2 Touchdown speed and the motor finding

True-state training run (2M steps)	Success at $c = 1$	Median touchdown speed	Landings at ≤ 1 m/s
Vertical limit ± 3 m/s	91.4%	-3.36 m/s	0.4%
Vertical limit ± 1 m/s (default)	81.5%	-2.30 m/s	1.2%
Text reading of the speed penalty	30.5%	-1.66 m/s	2.5%
Trained with symmetric slow motors	89.7%	-1.93 m/s	4.2%

The same ± 1 m/s checkpoint flown with different motor models (evaluation only): touchdown -2.30 m/s with AerialGym's motors, -1.52 m/s with symmetric slow motors, -0.78 m/s with symmetric fast motors, while its commanded descent near touchdown stayed at about -0.3 m/s. With fast symmetric motors, 136 of 1,000 episodes never came down within 30 s.

- **Why:** AerialGym's motors slow down 10–16 times faster than they speed up. When commands change every 0.1 s, the delivered thrust averages below what was asked (measured 12.71 N delivered against 14.06 N commanded), and with no integral term in the vertical controller the drone sinks at about 0.7 m/s without any descent command.
- **The policies exploit this:** they command little descent and let the drone sink.
- **But it is not the root cause:** training on symmetric motors still gave -1.93 m/s. The root cause is the task definition: the reward pays for descending, the +10 at touchdown replaces any speed penalty on that step, and success has no speed limit.
- **Decision:** keep the paper's motors and reward; report safe success next to the paper's success.

8.3 Vision configuration A, three seeds

Run	Training	Final curriculum level	Wall time
A_s1	4M steps	50 ($c = 0.625$)	20.0 h
A_s2	4M steps	50 ($c = 0.625$)	20.1 h
A_s3	4M steps	30 ($c = 0.375$)	20.3 h
P seeds 1–3 (for comparison)	2M steps	80 ($c = 1.0$), all three	0.3–0.4 h

Difficulty c	P seeds 1 / 2 / 3	P mean	A seeds 1 / 2 / 3	A mean	A position-estimate error (seed 1)
0.125	100 / 100 / 99.7%	99.9%	93.4 / 93.8 / 90.1%	92.4%	0.31 m

Difficulty c	P seeds 1 / 2 / 3	P mean	A seeds 1 / 2 / 3	A mean	A position-estimate error (seed 1)
0.5	99.0 / 99.2 / 99.3%	99.2%	88.8 / 91.1 / 67.9%	82.6%	0.84 m
0.625	97.5 / 97.6 / 97.1%	97.4%	82.2 / 83.8 / 56.9%	74.3%	1.37 m
1.0	81.5 / 89.9 / 83.8%	85.1%	48.5 / 52.4 / 34.7%	45.2%	4.36 m

- At $c = 0.5$, A's estimation error (0.68–0.84 m over seeds) is close to the paper's 0.95 m for this configuration.
- The position-estimate error grows with time out of view: about 0.2 m while the pad is visible, 1–3 m after 16–60 blind steps, and 6.7 m at full difficulty.
- A lands at -3.5 to -4.5 m/s (median), even harder than P; safe landing at ≤ 1 m/s is about 0%.

8.4 Without the estimation loss (A-noEst, one seed)

c	A mean (3 seeds)	A-noEst
0.125	92.4%	25.2%
0.5	82.6%	7.9%
0.625	74.3%	7.1%
1.0	45.2%	4.5%

A-noEst stayed at level 10 for all 4M steps and mostly crashed into the ground or flipped, even at the easiest level. The paper reports the same direction (73.99% vs 91%), but the effect is far larger here, plausibly because the plain CNN has no pre-trained features to start from, so the state supervision is what teaches it what to look at (untested).

8.5 True-state injection, three seeds

c	A mean	A with true state injected (mean)	Gap from P mean to A mean	Part of the gap recovered
0.5	82.6%	84.8%	16.6 points	13% (per seed 3–16%)
0.625	74.3%	77.3%	23.1 points	13% (per seed 3–27%)
1.0	45.2%	50.4%	39.9 points	13% (per seed 4–20%)

The weakest seed (A_s3) gains least from the true state. If a wrong estimate were the main cause of failure, it should gain most.

8.6 Curriculum-matched comparison

P saved a checkpoint the first time it reached each level. level_60.pt was therefore trained through level 50, the level A_s1 and A_s2 finished at; level_40.pt was trained through level 30, where A_s3 finished.

c	P trained through level 50 (mean of 3)	A at level 50 (A_s1, A_s2 mean)	P trained through level 30 (mean of 3)	A_s3 at level 30
0.5	88.0%	90.0%	77.7%	67.9%
0.625	84.0%	83.0%	72.3%	56.9%
1.0 (beyond training)	70.7%	50.5%	59.2%	34.7%

At the same stage, on the platform motion it has trained on, vision lands as well as the true-state policy. Most of the earlier gap at $c \leq 0.625$ came from P having trained further. Beyond the trained motion ($c = 1$), vision is about 20 points worse than the matched true-state policy.

8.7 Learning speed

Policy	Steps to reach level 40	Steps to reach level 60	Level after 4M steps
P seed 1	446,464	532,480	80 (reached within 2M)
P seed 2	315,392	389,120	80
P seed 3	495,616	606,208	80
A seeds 1 / 2 / 3	about 3.2M (seed 1)	not reached	50 / 50 / 30

The true-state policy passes level 40 after 0.3–0.5 million steps; vision needs about 3.2 million (seed 1) and one seed had not reached it after 4 million. **Vision needs roughly 7–10 times more training experience** to reach the same stage.

9. Conclusions and their limits

State or control? The difficulty lies on the state side, but in a specific form: (1) **learning** to extract the platform's state from images is slow (7–10 times more experience) and fragile (one seed out of three lagged far behind), and impossible here without explicit state supervision; (2) **once learned**, vision lands as well as a true-state controller at the same training stage on the motion it was trained on, and correcting its explicit estimate at test time fixes only about 13% of its shortfall; (3) on **faster, unseen** platform motion it generalises worse, and its estimate degrades while the pad is out of view.

Second finding: all learned policies land too hard (median 2–4.5 m/s, at most 5% of landings at ≤ 1 m/s) although gentle landing is physically possible (the oracle lands at 0.6 m/s). The paper's success measure counts these as successes, which supports the thesis claim that success rate alone is misleading.

Limits of these conclusions:

- Injection replaces only the 6-number estimate; the decision layer also reads 250 other internal numbers derived from the image, which may carry state information. A stronger test would retrain only the decision layer on the true state.
- A-noEst has one seed; A has only two seeds at level 50.
- Curriculum-matched checkpoints are snapshots at promotion, not after equal training time; P's matched results vary strongly by seed (76–96% at $c = 0.5$).
- Everything uses the ArUco pad, not the paper's keypoint pad, and no vision run has reached full difficulty, so the paper's 91% cannot yet be compared directly.
- The simulator is PyBullet with AerialGym's parameters, not AerialGym itself.

10. Next steps (each needs approval)

Step	Cost	Answers
Continue A_s2 for about 4M more steps so it reaches full difficulty	about 20 h	Comparison with the paper's 91% and with P at $c = 1$.
Retrain only A's decision layer on the true state, all else frozen	about 1 day of work plus a run	Removes the 250-number caveat of injection.
A-noEst seeds 2 and 3	about 40 h	Rule D14 for the ablation.
Baseline H: marker detector + Kalman filter + RL	build plus runs	Learned versus model-based estimation.
PACMAN keypoint pad (when weights are available)	build plus runs	The paper's full method (97%).
A safe-landing variant (speed limit in success or an impact penalty)	runs	Whether gentle landings can be learned.
Semester 2: transfer to the lab's ArduPilot drones	–	The long-term goal.

11. Every file: path and purpose

Repository root on the lab desktop: ~/mtp/drone-landing-rl/. All version-5 work is in v5_shin/; the paths below are relative to ~/mtp/drone-landing-rl/v5_shin/. Nothing outside it is changed except .gitignore, which keeps checkpoint files out of version control.

11.1 Documents and assets

Path	Purpose
SPEC.md	The master specification and lab notebook: every paper parameter with a status tag, decisions D1-D18, per-block test results, and every run and evaluation with checkpoint, step count and difficulty.
reports/	Status reports in PDF (this report is the most complete).
assets/lmf2.urdf	Drone model file: one merged rigid body, 1.24 kg, inertia, 0.5 m collision cube.
assets/ground_tex/	50 generated ground textures used for domain randomisation.
assets/pad_box.obj, assets/ground_quad_s*.obj	Generated meshes: the pad box with the marker on top; one ground mesh per texture scale.

11.2 Simulation and environment: envs/

Path	Purpose
envs/lmf2_params.py	Drone constants (mass, inertia, motor limits and time constants), controller gain ranges and their random sampling, timing (100 Hz physics, 10 physics steps per decision).
envs/lee_controller.py	Lee geometric velocity controller (port of AerialGym's): velocity and turning-rate command → total thrust and three torques.
envs/quad.py	Drone in PyBullet: motor model (with diagnostic modes), applying forces and torques, disturbances, reading the state; creating the PyBullet connection (headless, GPU renderer or 3D window).
envs/platform.py	Moving platform: random speed and turning rate, curriculum scaling, pad geometry.
envs/camera.py	Onboard camera: position and tilt, grayscale image, colour scaling and light direction, projection of 3D points into the image.
envs/pad_marker.py	Generates the ArUco texture and pad mesh; marker corner geometry; OpenCV marker detector (tests and future baseline).
envs/ground.py	Textured ground with random texture, scale and brightness.
envs/dr.py	Domain-randomisation settings and samplers; sensor-noise model.
envs/landing_sim.py	Simulation core: seeded random streams, start sampling, physics stepping, outcome detection, touchdown logging.
envs/shin_env.py	Standard reinforcement-learning environment interface (Gymnasium): observations, action scaling, which inputs each policy may see.
envs/reward.py	The paper's reward, with both readings of the vertical-speed penalty.
envs/curriculum.py	Curriculum levels and promotion rule.

11.3 Learning: policies/

Path	Purpose
policies/encoder.py	Image downsampling and the CNN.
policies/shin_policy.py	The complete network (CNN, LSTM, estimation layer, decision layer, critic), the true-state actor for variant P, and the true-state injection switch.
policies/ppo.py	Recurrent PPO training: collecting experience, advantage estimation, losses including the estimation loss, gradient accumulation, checkpoints.
policies/active_perception.py	The active-perception reward.
policies/oracle.py	The scripted oracle controller and a random policy.

11.4 Scripts you run: scripts/

Path	Purpose
scripts/train_ppo.py	Train a policy (any variant); writes everything to runs/<name>/.
scripts/evaluate.py	Pinned evaluation of a checkpoint; diagnostics --motor and --inject-true-state.
scripts/watch.py	Watch landings: live 3D window or video file, for any run or the oracle.
scripts/eval_oracle.py	Evaluate the scripted oracle or a random policy.
scripts/bench_env.py	Measure simulation and rendering speed.
scripts/gpu_update_check.py	Measure GPU memory and time of one training update.
scripts/reward_profile.py	Compute the reward collected by fixed descent rates, hovering and the oracle.
scripts/probe_encoder.py	Supervised probe: can the CNN locate the pad from one image?

11.5 Tests: tests/ (178 tests, all passing)

Path	Number	What it checks
tests/test_block1_controller.py	31	Mass and inertia, hover, speed steps, heading frame, turning limit, motor time constants, the documented sinking effect.
tests/test_block2_platform.py	10	Platform motion and its statistics.
tests/test_block3_camera.py	18	Camera geometry, marker detection and corner accuracy with both renderers.
tests/test_block4_termination.py	12	Every outcome type, start positions, success region.
tests/test_block5_oracle.py	4	Solvability with the oracle; random-policy reachability.
tests/test_block6_dr.py	17	Randomisation statistics, sensor noise, randomised appearance.
tests/test_block7_encoder.py	5	Downsampling, CNN shapes and gradients, learning a tiny dataset.
tests/test_block8_env.py	12	Environment interface, observations, privilege boundary, action limits.
tests/test_block9_reward.py	19	Every reward term computed by hand.
tests/test_block10_curriculum.py	9	Promotion rule and saving its state.
tests/test_block11_network.py	10	Network dimensions, privilege boundary, memory handling, which loss trains which part.
tests/test_block12_ppo.py	10	Advantage estimation, timeouts, recurrent replay, gradient accumulation, training command line.
tests/test_block13_active.py	8	Active-perception reward checked by hand on recorded experience.
tests/test_block14_evaluate.py	3	Evaluation is identical with any number of worker processes; vision metrics.
tests/test_diag_motor.py	5	Motor modes leave every random stream unchanged; sinking by mode; flags.
tests/test_diag_inject.py	2	True-state injection changes only the estimate input.
tests/test_watch.py	3	The viewer writes a video for a true-state run, a vision run and the oracle.

12. Where results are stored and in what form

12.1 One folder per training run: runs/<name>/

File	Form and content
config.json	Text (JSON): every training option (mode, seed, total steps, λ_{est} , r_{active} , vertical limit, reward reading, motor, renderer...), the PPO settings, the machine name and software versions.
updates.csv	Table, one row per PPO update (4,096 decisions): update, global_step, wall_s, sps, rollout_s, learn_s, level, c, window_n, window_success, episodes, n_success, n_crash_ground, n_crash_platform, n_tilt, n_drift, n_timeout, strict_success, impact_vz_mean, ep_len_mean, reward_mean, r_active_mean, r_active_frac, losses_pg (policy), v (value), ent (entropy), est (estimation), kl, clipfrac, explained_var, lr, log_std_mean; for vision also the estimation error by blind age.
episodes.csv	Table, one row per training episode: update, level, outcome, steps, c_episode, com_over_pad, impact_vz, impact_speed, tilt_deg.
curriculum.csv	Table, one row per 512-episode window: total_episodes, level, window_success, promoted.
latest.pt, u00050.pt, u00100.pt, ..., level_20.pt ... level_80.pt	PyTorch checkpoint files (binary, not in git): network weights (policy), optimiser state (opt), update number, global_step, settings (cfg) and the curriculum state (extra). latest.pt is overwritten every update; uNNNNN.pt every 50 updates; level_LL.pt the first time each level is reached.

12.2 One folder per evaluation, inside the run folder

Name: eval_<checkpoint>_c<difficulty>_s<seed base>_det[_<motor>][_injtrue]/, for example runs/A_s1/eval_latest_c0.5_s9000_det_injtrue/.

File	Form and content
summary.json	Text (JSON): run, checkpoint, timestep, difficulty, number of episodes, motor, inject flag; success_rate and success_ci95; strict_success_rate; outcomes (counts per type); impact_vz_median, _p10, _p90; impact_speed_median; steps_to_land_median; safe_success and safe_strict_success at 0.5 / 1.0 / 1.5 / 2.0 m/s; cmd_vz_last5_median; for vision est_pos_rmse_m, est_vel_rmse_ms and est_pos_err_by_blind_age.
episodes.csv	Table, one row per test episode: seed, outcome, steps, ret (return), com_over_pad, impact_vz, impact_speed, touch_x_pad, touch_y_pad, tilt_deg, dz0 and d0 (start height and distance), v_plat0, cmd_vz_last, cmd_vz_last5.

12.3 Other outputs

Path	Content
runs/<name>.log	Console output of a training run (one line per update).
runs/<name>.eval.log, .inj.log	Console output of the evaluations in the training queues.
runs/bench_<timestamp>/bench.csv	Speed benchmark table.
runs/<name>/watch_*.mp4 (when you create them)	Videos from the viewer.
reports/*.pdf	Status reports.
SPEC.md, section 15 (Runs log)	The written record and interpretation of every run and evaluation.

13. Commands

Run every command from ~/mtp/drone-landing-rl with the virtual environment active. Checkpoints exist only on the lab desktop (they are not in git), so the watch and evaluation commands must run there.

13.1 Set-up and tests

```
cd ~/mtp/drone-landing-rl
source drone-landing-rl-venv/bin/activate
python -m pytest v5_shin/tests -q          # expect: 178 passed
```

13.2 Watch each method land

A live 3D window opens and plays the first test episodes in real time (--slow 2 plays at half speed). Add --video FILE.mp4 to save a video instead (left: chase view with the drone circled in red; right: the drone's own camera). Open a video with xdg-open FILE.mp4. The console prints each episode's outcome and touchdown speed.

Method	Command
Scripted oracle	python -m v5_shin.scripts.watch --oracle --c 1.0 --episodes 3
True-state policy P	python -m v5_shin.scripts.watch --run P_smoke_vz1_s1 --c 1.0 --episodes 3
P, vertical limit ± 3 m/s (dives)	python -m v5_shin.scripts.watch --run P_smoke_s1 --c 1.0 --episodes 3
P, text reading of the penalty	python -m v5_shin.scripts.watch --run P_smoke_prose_vz1_s1 --c 0.125 --episodes 3
P trained on symmetric motors	python -m v5_shin.scripts.watch --run P_smoke_vz1_symslow_s1 --c 1.0 --episodes 3
P at a curriculum-matched checkpoint	python -m v5_shin.scripts.watch --run P_smoke_vz1_s1 --ckpt level_60.pt --c 0.625

Method	Command
Vision A (trained motion)	<code>python -m v5_shin.scripts.watch --run A_s1 --c 0.5 --episodes 3</code>
Vision A (full difficulty)	<code>python -m v5_shin.scripts.watch --run A_s1 --c 1.0 --episodes 3</code>
Vision A with the true state injected	<code>python -m v5_shin.scripts.watch --run A_s1 --c 1.0 --inject-true-state</code>
Vision A on symmetric fast motors	<code>python -m v5_shin.scripts.watch --run A_s1 --c 0.5 --motor sym_fast</code>
A without the estimation loss	<code>python -m v5_shin.scripts.watch --run A_noEst_s1 --c 0.125 --episodes 3</code>
Save any of these as a video	<code>... --video v5_shin/runs/A_s1/watch_c0.5.mp4</code>

13.3 Train each method

```
# true-state policy P (about 25 min)
python -m v5_shin.scripts.train_ppo --mode privileged --name P_smoke_vz1_s1 --seed 1 \
    --total-steps 2000000 --renderer tiny
# vision A (about 20 h); run in the background so closing the terminal does not stop it
nohup python -m v5_shin.scripts.train_ppo --mode vision --name A_s1 --seed 1 --total-steps 4000000 \
    --renderer tiny --device cuda > v5_shin/runs/A_s1.log 2>&1 &
# A without the estimation loss: add --lambda-est 0
# A without active perception: add --r-active off
# other options: --vz-max 3.0, --vz-penalty prose, --motor sym_slow, --resume (continue a stopped run)
```

13.4 Watch training progress

```
tail -f v5_shin/runs/A_s1.log | grep --line-buffered -E "^u"
# one line per update; Ctrl+C stops watching only (training continues)
# columns: update, steps, level L, window successes, episodes, successes, crashes,
#   tilts, drift, timeouts, kl, explained variance, est (estimation loss),
#   ra (active-perception term), speed
```

13.5 Evaluate and diagnose

```
# pinned evaluation (true-state policy; seconds)
python -m v5_shin.scripts.evaluate --run P_smoke_vz1_s1 --ckpt latest.pt \
    --episodes 1000 --c 0.5 0.625 1.0
# vision policy (minutes; must use the CPU renderer it was trained with)
python -m v5_shin.scripts.evaluate --run A_s1 --ckpt latest.pt \
    --episodes 1000 --c 0.5 1.0 --renderer tiny
# diagnostics
python -m v5_shin.scripts.evaluate --run A_s1 --c 1.0 --renderer tiny \
    --inject-true-state
python -m v5_shin.scripts.evaluate --run P_smoke_vz1_s1 --c 1.0 --motor sym_fast
# scripted oracle
python -m v5_shin.scripts.eval_oracle --policy oracle --c 1.0 \
    --episodes 200 --seed-base 9000
```

13.6 Read a result

```
python - <<'EOF'
import json
s = json.load(open('v5_shin/runs/A_s1/eval_latest_c0.5_s9000_det/summary.json'))
print(s['success_rate'], s['success_ci95'], s['outcomes'])
print(s['impact_vz_median'], s['safe_success'])
EOF
```

14. Viva questions with answers

Ordered from basic to advanced. Answers are written to be spoken in one or two minutes.

14.1 Basic

Q1. What problem does your project solve?

Landing a quadrotor on a moving ground vehicle using only the drone's own camera and motion sensors, with no GPS and no communication with the vehicle. The vehicle's speed and route change randomly and are unknown to the drone.

Q2. Why not simply use GPS on both vehicles?

GPS is often unavailable or inaccurate (near buildings, ships, indoors, under jamming), and the vehicle may not cooperate (it may not share its position). A camera is light and cheap and measures the relative position directly, which is what landing needs.

Q3. What is reinforcement learning, in one paragraph?

A method in which an agent learns a policy by acting, receiving a numerical reward, and adjusting the policy so as to collect more reward over time. Here the agent is the drone, the actions are velocity commands, and the reward favours approaching and touching the pad and punishes crashes.

Q4. What does the drone observe, and what does it control?

It observes a grayscale camera image and its own body-frame velocity and orientation, both with noise. It outputs desired forward, sideways and vertical speeds and a turning rate, ten times per second; a geometric flight controller converts them into motor thrusts.

Q5. Why is this problem "partially observable"?

The platform's position and velocity are not measured directly; they must be inferred from images, and the pad is often outside the image. A single image cannot tell the platform's velocity, so memory is required.

Q6. What is a curriculum and why use one?

Training starts with a slow, predictable platform and makes it faster and more erratic only after the policy succeeds 80% of the time. Starting directly at full difficulty gives almost no successes to learn from.

Q7. What is domain randomisation and why is it used?

Randomly varying physical and visual properties during training (controller gains, wind-like forces, initial motion, ground texture, lighting, colours). It prevents the policy from relying on details of one simulated world and is meant to help transfer to the real drone.

Q8. Why ArUco instead of the paper's keypoint pad?

The paper's keypoint detector (PACMAN) has no public trained weights. The paper itself reports an ArUco + plain CNN variant (91%), so that is a fair, published target. The keypoint pad will be added if the weights become available.

Q9. How do you define success? What are strict and safe success?

Success follows the paper: any contact with the pad's top surface. Strict success also requires the drone's centre to be over the pad. Safe success also requires the touchdown speed to be at most a threshold such as 1 m/s.

Q10. Why simulation, and why PyBullet rather than the paper's AerialGym?

Training needs millions of attempts, which is impossible on hardware. AerialGym runs on Isaac Gym, which needs hardware and software we do not have; PyBullet runs on the lab desktop. We copied AerialGym's drone, controller and motor parameters exactly so that the dynamics match as closely as possible.

14.2 Intermediate

Q11. Explain PPO.

PPO alternates between collecting experience with the current policy and improving it. For each action it estimates an advantage (how much better the outcome was than the critic predicted). It then increases the probability of actions with positive advantage, but limits the change: the probability ratio between new and old policy is clipped to $[0.8, 1.2]$, so updates stay small and stable.

Q12. Why an LSTM?

Because the pad is out of view about half of the time and its velocity cannot be read from a single image. The LSTM keeps a memory of past images and motion, so the policy can continue to track the platform while it is unseen.

Q13. What is the asymmetric critic and is it cheating?

During training the critic (value network) sees the true platform state, which makes its predictions and therefore the learning signal more accurate. It is not cheating, because the critic is used only during training in simulation; the deployed policy never sees privileged information. Tests verify that the policy's output does not depend on it.

Q14. What is the estimation loss and why does it help?

The first 6 numbers of a hidden layer are trained to equal the true relative position and velocity of the platform. This gives the image-processing part of the network a direct, dense signal about what matters, instead of relying only on the sparse landing reward. Our A-noEst ablation shows that without it the vision policy does not learn at all.

Q15. What is the active-perception reward?

A small penalty, up to 0.1 per step, when the network's estimation error at the next step exceeds 0.01. It rewards flying in a way that keeps the platform well estimated, usually by keeping it in view. In our runs the threshold 0.01 is far below the actual error, so the term is active almost always and acts as a penalty proportional to the error.

Q16. Explain the vertical-speed penalty problem.

The paper prints $-[v_z + 0.5]^+$, which penalises everything slower than a 0.5 m/s descent, including hovering, while its text says it penalises fast descent. We proved that no sign convention reconciles them, tested both, and kept the printed version because the text version made learning stall (30.5% vs 81.5%).

Q17. What are γ and λ , and why do timeouts need special handling?

$\gamma = 0.99$ discounts future rewards; $\lambda = 0.95$ trades bias against variance in advantage estimation. An episode that ends by timeout is not a real ending of the task, so the value of its last state is used instead of zero; otherwise the policy would learn that time running out is like crashing.

Q18. What is variant P and why is it called the control ceiling?

The same training (reward, PPO, curriculum) but the policy receives the true relative state instead of images. It shows the best a learned controller can do with perfect information. If P failed, the problem would be in control or training, not in perception.

Q19. Why pinned evaluation and confidence intervals?

Success rates vary with the random episodes drawn. Using the same 1,000 episodes for every method removes that source of difference, and the Wilson 95% interval (about ± 2 –3 points at 1,000 episodes) shows which differences are larger than chance.

Q20. Why limit the vertical command to ± 1 m/s?

The paper does not state the limit. With ± 3 m/s the policy dived at about 4 m/s; the paper's trajectory figures show descents of about 1 m/s. Decision D16 records this.

Q21. Why does the drone sink faster than it is commanded?

The simulated motors slow down 10–16 times faster than they speed up. When commands change rapidly, the delivered thrust is on average lower than requested, and the vertical controller has no integral term to correct a constant shortfall, so the drone sinks by about 0.7 m/s. Evaluating the same policy with symmetric motors removed most of the excess speed.

Q22. Why render on the CPU?

Each GPU-rendered environment reserves about 250 MB of graphics memory; 16 of them fill the 4 GB card and training then crashes. CPU rendering is slower per image, but the network update dominates the time, so overall speed is the same.

14.3 Advanced

Q23. How exactly did you separate state from control?

Four comparisons on identical test episodes. (1) True-state policy versus vision policy: the gap measures the cost of vision. (2) Injecting the true state into the vision policy at test time: if the gap closes, the explicit estimate was the problem. (3) Vision without the estimation loss: tests whether learning the state is necessary. (4) Comparing at the same curriculum stage, to remove the advantage of longer training.

Result: the vision policy matches the true-state policy at the same stage on motion it has trained on; injection fixes only about 13% of the remaining gap; without state supervision nothing is learned; and vision needs 7–10 times more experience. So the difficulty is learning the state from images, not acting on a learned estimate.

Q24. What are the limitations of the true-state injection test?

It replaces only the 6 explicit estimate numbers. The decision layer also reads 250 other numbers computed from the image, which may contain state information with its own errors. The decision layer was also never trained on exact true-state inputs. A cleaner test retrains only the decision layer on the true state with everything else frozen.

Q25. Why is A-noEst so much worse than the paper's "without state estimation" result?

The paper's ablation uses the keypoint pad, whose pre-trained keypoint encoder already provides strong geometric features; removing the estimator then costs about 17 points. On the ArUco base our CNN starts from scratch, so the estimation loss is likely its main signal about what to look at; without it, reward alone is too sparse. This explanation is plausible but untested, and A-noEst has one seed.

Q26. Why was the curriculum-matched comparison necessary, and what did it show?

The true-state policy finished the curriculum within 2M steps while the vision policies stopped at levels 50 and 30 after 4M. Comparing final checkpoints mixes the cost of vision with the benefit of more advanced training. Using P's checkpoints saved at the same level, vision equalled P on trained motion (90.0% vs 88.0% at $c = 0.5$, 83.0% vs 84.0% at $c = 0.625$) but was 20 points worse on faster, unseen motion.

Q27. Are your results statistically meaningful?

Within one checkpoint, 1,000 episodes give intervals of about ± 2 –3 points. Across training runs the variation is larger: for example, vision seeds reached levels 50, 50 and 30. The main comparisons therefore use three seeds for P and A and report the spread; A-noEst and some matched comparisons rest on fewer seeds, which is stated.

Q28. Why does vision learn so much more slowly?

The vision policy must learn to extract the platform's position and velocity from pixels and to remember it while the pad is out of view, in addition to learning to fly. The true-state policy receives that information directly. The estimation loss provides the perception signal, but it still takes millions of images, and one seed in three lagged far behind.

Q29. Your landings hit at 2-4 m/s. Does that invalidate the reproduction?

No; it shows a weakness of the task definition. The paper's success measure has no speed condition, the reward pays for descending, and the +10 at touchdown replaces any speed penalty on that step. We reproduce the paper's definition and additionally report safe success. The oracle proves gentle landing is possible (0.6 m/s), so a safe-landing variant is a planned separate experiment.

Q30. What deviations from the paper remain, and how do they affect conclusions?

PyBullet instead of AerialGym (same drone and controller parameters; motor integration differs slightly, effect below 0.06 m/s), ArUco instead of the keypoint pad, our own choices where the paper is silent (D1-D18, all recorded), CPU rendering, and our vision runs not yet reaching full difficulty. The state-versus-control conclusions are internal comparisons on identical episodes, so these deviations affect all variants equally.

Q31. What does this mean for the real drone in semester 2?

The learned policies exploit a property of the simulated motors (fast spin-down) to descend; real motors and the PX4 flight controller behave differently, so touchdown behaviour may change. Before transfer, the motor response of the lab drones should be measured and the simulator matched, and a safe-landing criterion should be added.

Q32. Why does the true-state policy reach only 85% at full difficulty?

Even with perfect information the task is hard: the platform can reach 8 m/s and turn at 30°/s, the drone's controller responds with a lag, and the reward encourages fast descents that occasionally hit the ground or the platform's side. The three seeds spread from 81.5% to 89.9%.

Q33. Why does the vision policy land harder than the true-state policy?

Its commands are jerkier, which strengthens the motor sinking effect; with symmetric fast motors its touchdown speed drops from about -3.5 to -1.6 m/s, but then most episodes drift away, which shows how much it relies on that effect.

Q34. How do you know the simulator is correct?

178 automated tests check every block: mass and inertia, controller responses against theory, platform statistics, camera geometry to sub-pixel accuracy, outcomes, rewards computed by hand, network wiring and learning mechanics. The scripted oracle lands 99% of the time at full difficulty, and several real bugs (inertia, rendering, impact speed) were caught by these tests.

Q35. How did you prevent the vision policy from accidentally seeing the true state?

The environment computes all quantities but a fixed table decides what each policy may read. Tests change only the hidden platform velocity and verify that the vision policy's inputs and outputs are bit-identical, and that the estimation loss trains only the perception and estimation parts.

Q36. What would you do next and why?

Continue a vision run to full difficulty to compare with the paper's 91%; retrain only the decision layer on the true state to remove the main caveat of the injection test; add seeds for A-noEst; build the Kalman-filter baseline; and test a safe-landing variant before moving to real hardware.

Q37. If the state is the problem, why not use a classical estimator (Kalman filter)?

That is the paper's EKF + RL baseline (59.77% in the paper). It relies on detecting the marker; when the marker is lost it can only predict with a constant-velocity model, which fails when the platform accelerates. The learned estimator can use partial and indirect visual cues. Building this baseline is in the plan, to test the comparison in our setting.

Q38. What is the single most important lesson of this work so far?

That a single success number hides the mechanism. The same success rate can come from very different behaviour; the gap between two methods can come from training stage rather than capability; and counted successes can be unsafe impacts. Controlled comparisons on identical episodes, at matched training stage, with failure types and touchdown speeds, were needed to answer the state-versus-control question.